



National University of Modern Languages, Islamabad

LAB EXAM

Submitted To:	Sir Shahid
Submitted By:	Tayyaba Ejaz
Roll Number:	2751
Subject:	MAD
Due Date:	22th December 2025
Semester:	8th
Course:	BSCS, Morning

Faculty of Engineering and Computer Science

Table of contents

1. Lab Task for Attendance App	3
1.1 Code	4
1.1.1 Model: alert_bloc.dart	4
1.1.2 Screen: library_screen.dart	4
1.1.3 Provider: library_provider.dart	9
1.2 Screenshots.....	10

1. Lab Task for Attendance App

Develop a Flutter application that demonstrates state management using the Provider package by implementing a simple library book availability tracker. The application should display a predefined list of books and allow the user to manage their availability status within the app. Each book record must contain an ID, a title, and a boolean flag indicating whether the book is currently available or issued. No external APIs or databases are required; all data should be managed within the application state.

Create a Provider class named `LibraryProvider` that extends `ChangeNotifier`. This provider should maintain a list of at least five predefined book objects. It must include methods to toggle the availability status of a selected book, mark all books as available, and mark all books as issued. A computed getter should also be implemented to return the total number of available books. Every change in the book list or book status must correctly call `notifyListeners()` so that the user interface updates automatically.

The user interface should consist of a single screen titled “Library Tracker.” At the top of the screen, display the total count of available books, which must update dynamically whenever a book’s status changes. Below this, use a scrollable list to show all books. Each list item should display the book title, its current status (Available or Issued), and a switch widget that allows the user to change the availability of that book. At the bottom of the screen, provide two buttons: one to mark all books as available and another to mark all books as issued.

The application must be wrapped with `ChangeNotifierProvider` at the root level. The user interface should use `context.watch<LibraryProvider>()` to listen for state changes and rebuild the necessary widgets, while `context.read<LibraryProvider>()` should be used for triggering actions such as button presses. The design should clearly demonstrate correct separation between the data model, provider logic, and user interface.

Students are required to submit the complete Flutter project along with screenshots showing multiple books with different availability states and the updated available books count.

1.1 Code

1.1.1 Model: alert_bloc.dart

```
class Book {  
  
    final int id;  
  
    final String title;  
  
  
    bool isExist;  
  
  
    Book({  
        required this.id,  
        required this.title,  
        this.isExist = false,  
    });  
}
```

1.1.2 Screen: library_screen.dart

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import '../library_provider/library_provider.dart';  
  
void main() {  
    runApp(  
        ChangeNotifierProvider(  
            create: (_) => LibraryProvider(),  
            child: const MyApp(),  
        ),  
    );  
}  
  
class MyApp extends StatelessWidget {
```

```
const MyApp({super.key});
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return MaterialApp(
```

```
    debugShowCheckedModeBanner: false,
```

```
    theme: ThemeData(
```

```
      scaffoldBackgroundColor: Colors.grey.shade100,
```

```
    ),
```

```
    home: const LibraryScreen(),
```

```
  );
```

```
}
```

```
}
```

```
class LibraryScreen extends StatelessWidget {
```

```
  const LibraryScreen({super.key});
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  final libraryProvider = context.watch<LibraryProvider>();
```

```
  return Scaffold(
```

```
    appBar: AppBar(
```

```
      title: const Text("Library Tracker"),
```

```
      centerTitle: true,
```

```
      elevation: 1,
```

```
    ),
```

```
    body: Column(
```

```
children: [
  /// Available Books Count
  Padding(
    padding: const EdgeInsets.all(16),
    child: Container(
      width: double.infinity,
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
      ),
      child: Text(
        "Available Books: ${libraryProvider.availableBooksCount}",
        style: const TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.w600,
        ),
        textAlign: TextAlign.center,
      ),
    ),
  ),
  /// Book List
  Expanded(
    child: ListView.builder(
      padding: const EdgeInsets.symmetric(horizontal: 12),
      itemCount: libraryProvider.books.length,
      itemBuilder: (context, index) {
```

```
final book = libraryProvider.books[index];

return Card(
  elevation: 1,
  margin: const EdgeInsets.symmetric(vertical: 6),
  child: ListTile(
    title: Text(
      book.title,
      style: const TextStyle(
        fontWeight: FontWeight.w500,
      ),
    ),
    subtitle: Text(
      book.isExist ? "Available" : "Issued",
      style: TextStyle(
        color: book.isExist
          ? Colors.green
          : Colors.red,
      ),
    ),
    trailing: Switch(
      value: book.isExist,
      onChanged: (_) {
        context
          .read<LibraryProvider>()
          .toggleBookAvailability(book.id);
      },
    ),
  ),
)
```

```

        ),
    );
},
),
),

/// Bottom Buttons
Padding(
  padding: const EdgeInsets.all(16),
  child: Row(
    children: [
      Expanded(
        child: ElevatedButton(
          onPressed: () {
            context
              .read<LibraryProvider>()
              .markAllAvailable();
          },
          child: const Text("Mark All Available"),
        ),
      ),
      const SizedBox(width: 12),
      Expanded(
        child: ElevatedButton(
          onPressed: () {
            context
              .read<LibraryProvider>()
              .markAllIssued();
          },

```

```

        },
        child: const Text("Mark All Issued"),
      ),
    ),
  ],
),
),
],
),
);
}
}

```

1.1.3 Provider: library_provider.dart

```
import 'package:equatable/equatable.dart';
```

```
import '../models/weather_alert.dart';
```

```

abstract class AlertState extends Equatable {
  @override
  List<Object?> get props => [];
}

```

```
// Initial State
```

```
class AlertInitial extends AlertState {}
```

```
// Loading State
```

```
class AlertLoading extends AlertState {}
```

```
// Loaded State
```

```
class AlertLoaded extends AlertState {
```

```
final List<WeatherAlert> alerts;
```

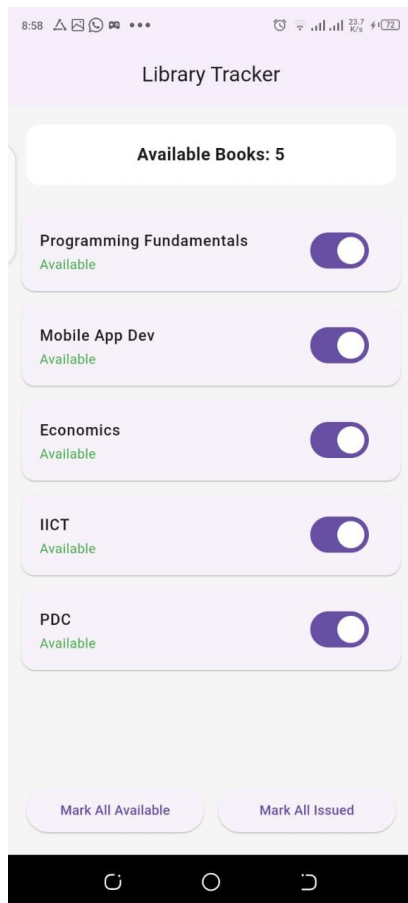
```
AlertLoaded(this.alerts);
```

```
@override
```

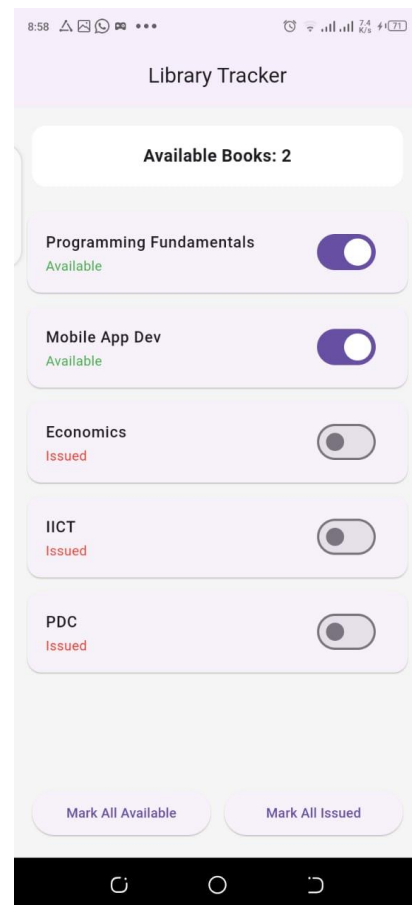
```
List<Object?> get props => [alerts];
```

```
}
```

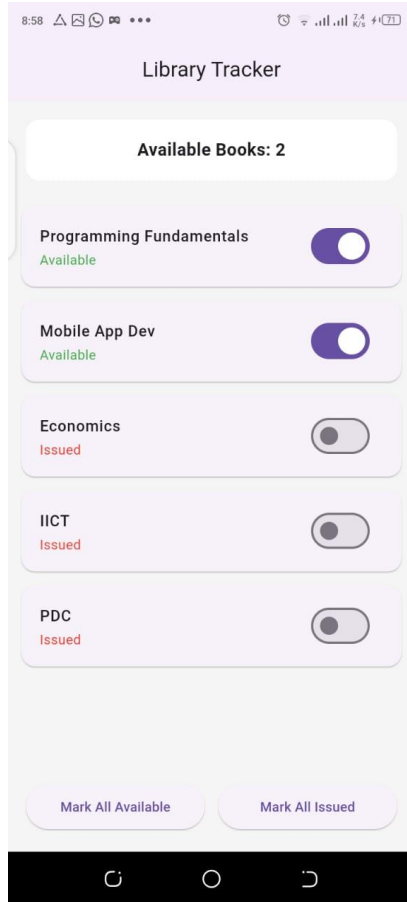
1.2 Screenshots



All Marked available



2 Available



All issued
